Vibe Coding 101

Week 3: Ship It Week

Deployment & CI/CD

Today's Agenda

1. **Prerequisites Check** — What you need before we start
2. **The 80/20 Rule** — Ship fast, iterate faster
3. **Tool Selection** — Choosing the right platform
4. **Secrets Management** — Keep your API keys safe
5. **TDD Light** — One test before you deploy
6. **The Sprint** — PRD → Deployed in 45 minutes

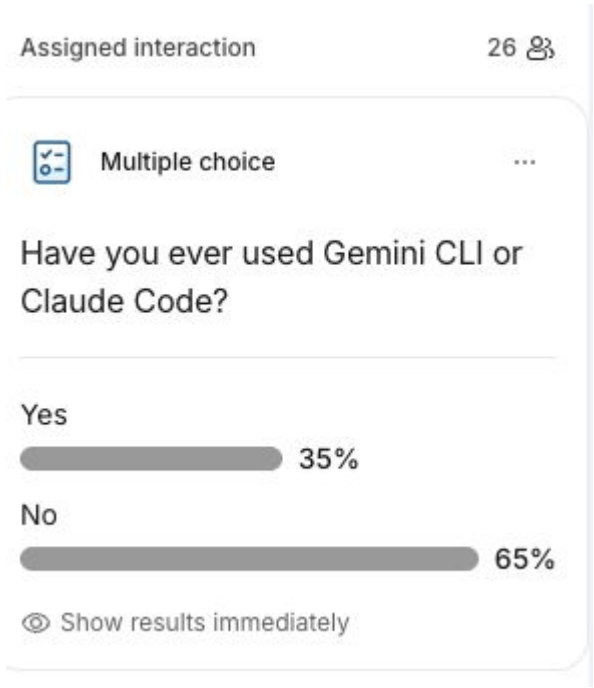
Pulse Check (Start)

Quick show of hands:

1. How many of you have used Gemini CLI or Claude Code?
2. Have you ever written a PRD?
3. Have you ever created a CLAUDE.md or GEMINI.md file?

If you answered "no" to any of these, today we fix that.

Poll results



Prerequisites Checklist

You MUST have these ready:

Requirement	How to Check
PRD Written	`docs/PRD.md` exists
AI Tool Installed	`claude --version` or `gemini --version`
CLAUDE.md Created	File in project root
Deployment Account	Can log into Vercel/Railway/Replit
Git Repo Ready	`git status` works, `.env` in `.gitignore`

If ANY box is unchecked, fix it NOW.

CLAUDE.md Example

```
# Project: Grocery List App

## Commands
- `npm run dev` – Start dev server
- `npm run build` – Production build
- `npm test` – Run tests

## Boundaries (NEVER)
- Never hardcode API keys
- Never skip tests
- Never deploy without build passing
```

Official Docs: docs.anthropic.com/en/docs/claude-code/overview

PRD Reminder

Section	Contains
Overview	What, Who, Why
Features	3-5 core features
Non-Goals	What you WON'T build
Phases	Step-by-step plan
Agent Rules	Always/Ask First/Never

Examples: `templates/examples/` (Todo CLI, Code Review, Weather API)

Learning Objectives

By the end of this class, you will:

1. **Execute** a rapid prototyping workflow: PRD → Deployed prototype
2. **Select** the right platform for your project type
3. **Configure** environment variables and manage secrets securely
4. **Debug** common deployment failures using logs and AI assistance
5. **Write** one test before deploying (TDD Light)

The 80/20 Rule

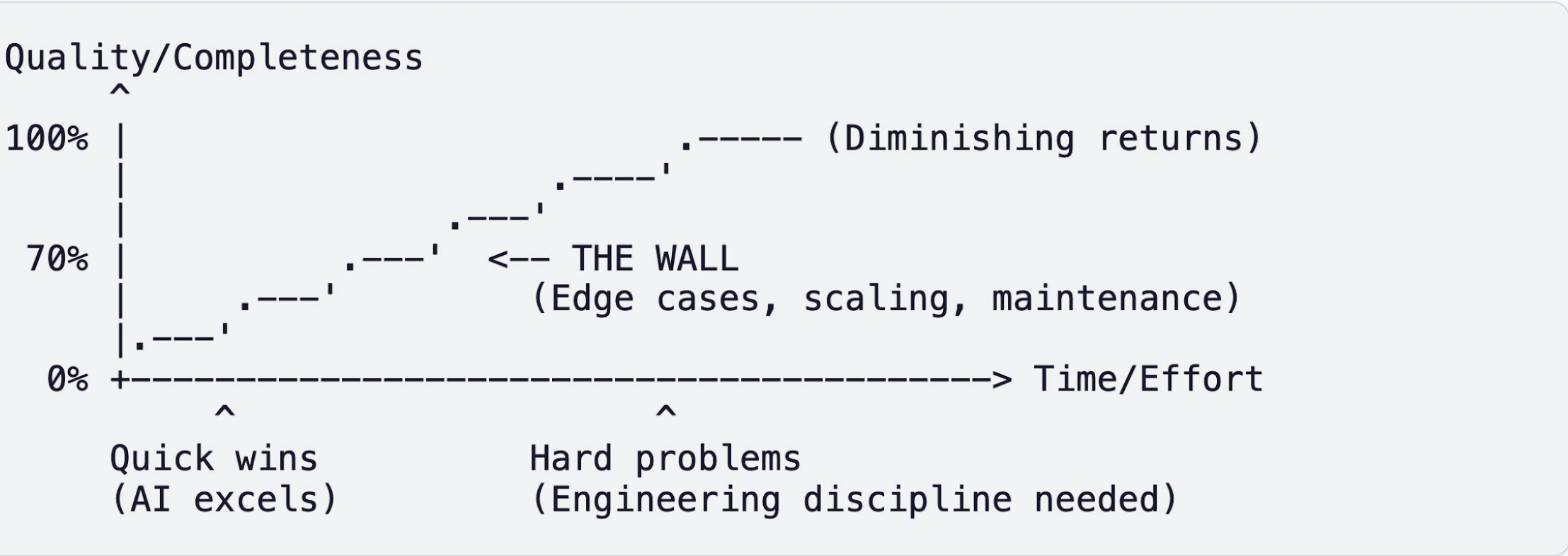
Get 80% working. Deploy immediately.

80% Done	20% Remaining
Core features work	Edge cases
UI is usable	Polish
Basic functionality	Robustness

Deployed 80% > Perfect 0%

Localhost perfectionists ship nothing. Real users are your best QA.

The 70% Wall



WEEKS 1-3: Get to 70% fast with AI

WEEKS 4-12: Push past the wall with discipline

The 70% Wall

WEEKS 1-3: Get to 70% fast with AI

WEEKS 4-12: Push past the wall with discipline

Visionary Quote

"Make something people want. Then launch early and often."

— Paul Graham, Y Combinator

Your users don't care about your code quality. They care about whether it solves their problem.

Tool Selection Matrix (All FREE Options)

Project Type	FREE Tools	Paid Tools
Web App	Bolt.new, v0, Replit	Cursor ($20/mo)
CLI Tool	Gemini CLI (FREE!), Claude Code	Claude Max
API/Backend	Replit, Windsurf	Cursor, Windsurf Pro
Mobile App	Expo (FREE) + any AI	Replit Mobile

Start free. Upgrade when you hit limits.

Gemini CLI is completely FREE with 1M token context!

Deployment Platforms (FREE Tiers)

Platform	Best For	Free Tier
Vercel	Next.js, React	100GB bandwidth
Railway	Any language	$5 credit/mo
Render	Simple apps	750 hours/mo
Replit	Learning	Free with ads
Netlify	Static sites	100GB

Students: Start with **Vercel** or **Replit** (zero config).

Secrets Management

The Golden Rule: Never commit `.env` files. Never put API keys in code.

macOS/Linux:

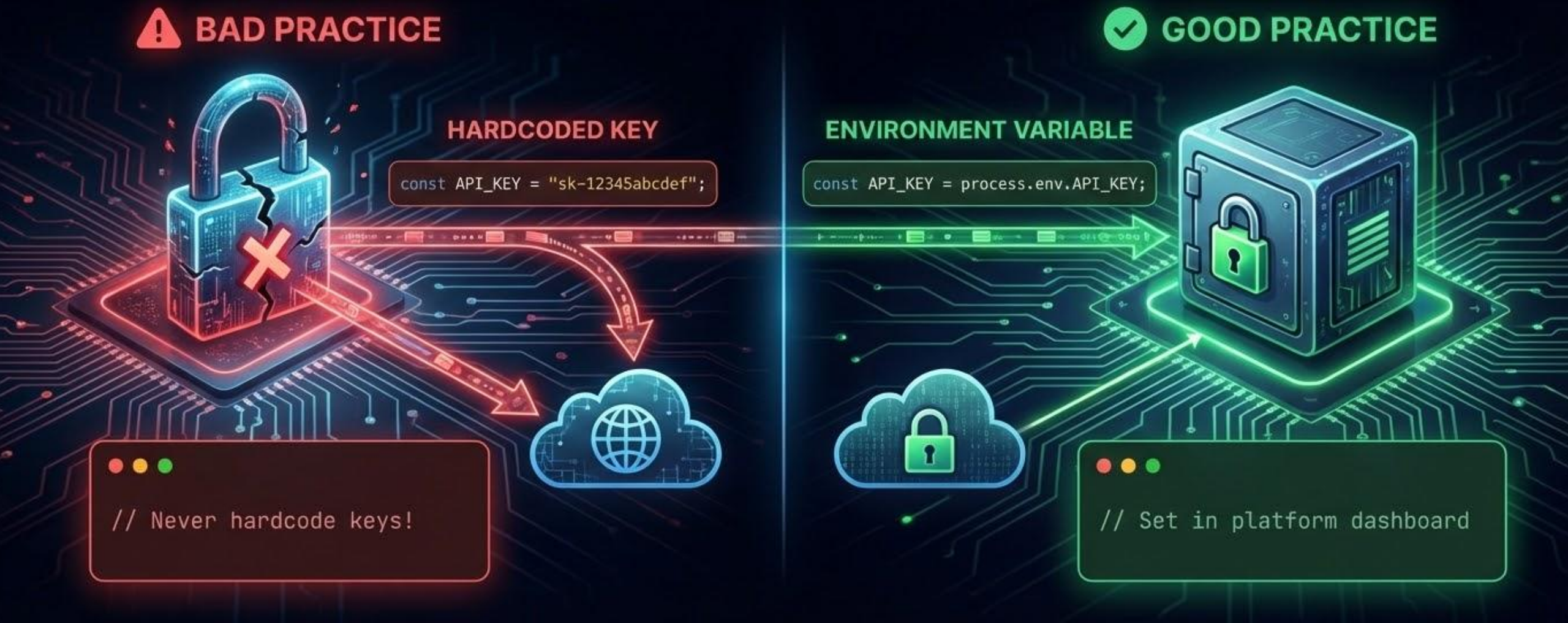
```
# Create .env file (git-ignored)
echo "OPENAI_API_KEY=your-key-here" >> .env
echo ".env" >> .gitignore
```

Windows PowerShell:

```
# Create .env file (git-ignored)
Add-Content -Path .env -Value "OPENAI_API_KEY=your-key-here"
Add-Content -Path .gitignore -Value ".env"
```

Never Commit API Keys

If you commit a key, assume it is compromised.
Always use Environment Variables.



The Deployment Checklist

Before you hit "Deploy":

Step	macOS/Linux	Windows
1. Works locally?	`npm run dev`	`npm run dev`
2. No secrets?	`grep -r "sk-" .`	`Select-String "sk-"`
3. Build passes?	`npm run build`	`npm run build`
4. ONE test passes?	`npm test`	`npm test`

Don't skip the test. One test > Zero tests.

The Sprint — Step by Step

You already have a PRD. Now execute.

Time	Task
0-5 min	Verify prerequisites, open PRD
5-15 min	AI generates initial version
15-25 min	Iterate: "Fix this bug", "Add dark mode"
25-30 min	Write ONE test (TDD Light)
30-40 min	Deploy to Vercel/Railway/Render
40-45 min	Share URL, get feedback

Why TDD? (Even One Test Matters)

The Problem:

- AI generates code that *looks* right but doesn't work
- 20% of AI-generated code has bugs you can't see
- Deployments can succeed but features can be broken

One Test Catches:

- "Core feature doesn't work" — before you embarrass yourself
- "API integration is broken" — before users see errors
- "Logic is wrong" — before you waste time debugging production

FREE Testing Tools

Tool	Language	Best For
Vitest	JavaScript/TS	React, Vue, modern JS
Jest	JavaScript/TS	Node, React
pytest	Python	All Python projects
Go test	Go	Go projects (built-in)

Install (all platforms):

```
npm install -D vitest   # JavaScript
pip install pytest      # Python
```

TDD Light — One Test Before Deploy

```
// grocery-list.test.js
import { test, expect } from 'vitest';
import { GroceryList } from './grocery-list';

test('user can add an item', () => {
  const list = new GroceryList();
  list.add('Milk');
  expect(list.items).toContain('Milk');
});
```

Run: `npm test`

If this test fails, your app is broken. Don't deploy broken apps.

TDD with AI — Red-Green-Refactor

Step	What to Do	AI Prompt
RED	Write failing test	"Write a Vitest test for adding items. It should fail."
GREEN	Make it pass	"Make this test pass with minimum code."
REFACTOR	Clean up	"Refactor to be cleaner. Keep test passing."

Works with Gemini CLI (FREE) or Claude Code.

This cycle takes 5-10 minutes. Do it ONCE before deploy.

Common Deployment Failures

Error	Cause	Fix
Build failed	Missing dep	Check logs, fix locally
Module not found	Wrong path	Verify path, `npm install`
Env var undefined	Not set	Add to platform settings
App crashes	Missing API key	Check logs, add key
504 Timeout	Too slow	Optimize cold start

Debugging with AI

When deploys fail, copy the error and ask:

Gemini CLI (FREE):

```
gemini "My Vercel build failed with this error:
[paste error log]

What's wrong and how do I fix it?"
```

Claude Code:

```
claude "Debug this deployment error: [paste error]"
```


AI is excellent at parsing error logs. Use it.

Pulse Check (End)

Quick review:

1. What platform will you deploy to? (Vercel / Railway / Render)
2. What's ONE test you'll write before deploying?
3. Where do you set environment variables in production?

Now go ship something!

Resources (All FREE)

Deployment: vercel.com, railway.app, render.com, replit.com, netlify.com

AI Tools:

- Gemini CLI (FREE, 1M context!): ai.google.dev/gemini-cli
- Bolt.new, v0.dev, claude.ai/code

Testing: vitest.dev, jestjs.io, pytest.org

All links in course README

What's Next

Week 4: The Explore → Plan → Code → Verify Cycle

- Context engineering for AI tools
- Instruction files (CLAUDE.md, .cursorrules)
- Git workflows for AI-assisted development
- Pushing past the 70% wall

You've shipped. Now we build discipline.

Instructor: Goker Ezberci | gokerez@gmail.com

Thank You!

Questions?

Week 3 Complete